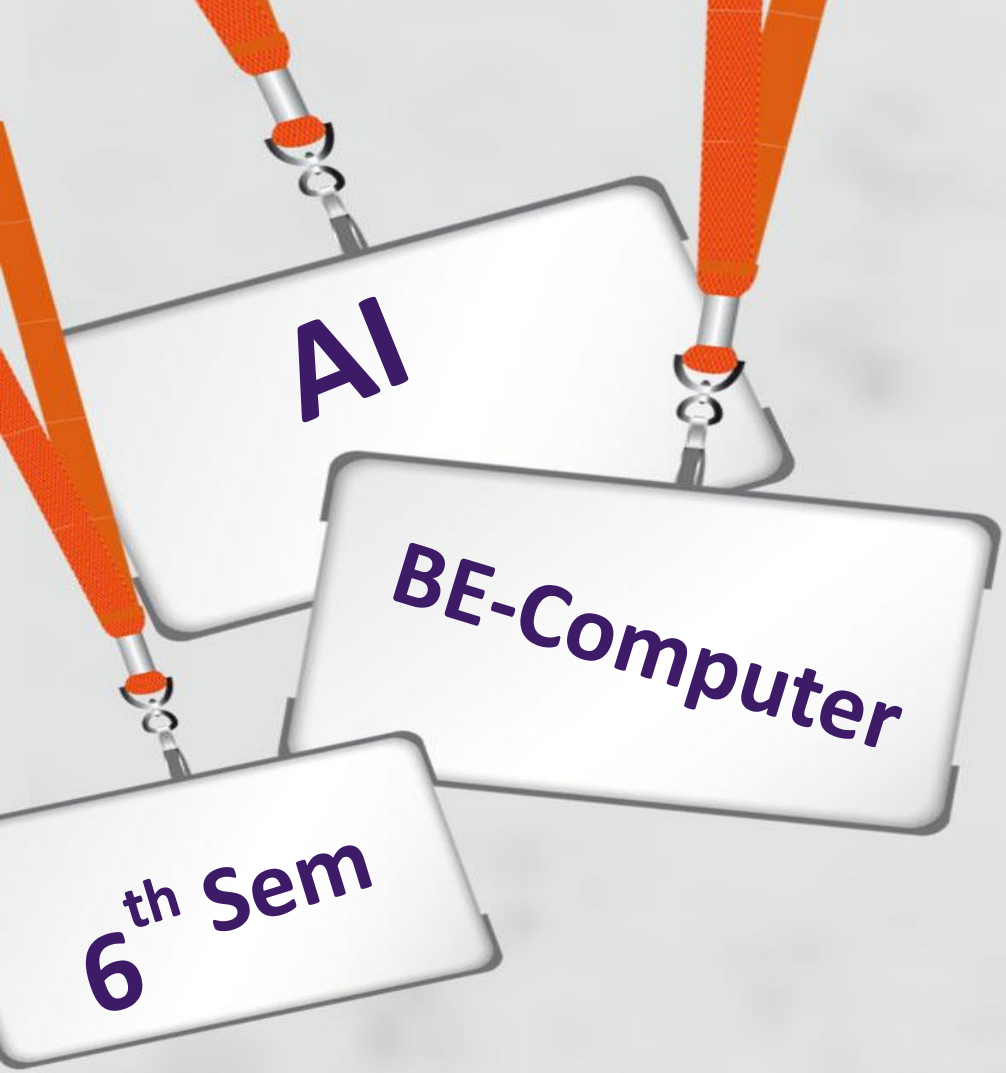


Chapter 7

Application of Artificial Intelligence



Just A Real Virtual Intelligent System

It recognize voice commands to do something, even for locking door and turn off the lamp.

MYCIN would attempt to diagnose patients based on reported symptoms and medical test results.

The program could request further information concerning the patient, as well as suggest additional laboratory tests, to arrive at a probable diagnosis, after which it would recommend a course of treatment.

If requested, MYCIN would explain the reasoning that led to its diagnosis and recommendation.

CALEX is a blackboard based integrated system for agricultural management, developed at University of California.

CALEX can be used by growers, pest control advisors, consultants and other managers.

DENDRAL's primary aim was to help organic chemists in identifying unknown organic molecules, by analyzing their mass spectra and using knowledge of chemistry.

The expert systems are the computer applications developed to solve complex problems in a particular domain, at the level of extra-ordinary human intelligence and expertise.

Expert Systems: Introduction

- Expert Systems are computer programs built for commercial application using the programming techniques of AI which are developed for problem solving
- Built for varieties of purposes including Medical Diagnosis, Electronic Fault Finding, Mineral prospecting, Computer System Configuration, etc.

Definition of Expert System:

An Expert System is a collection of programs or Computer Software that solves problems in the domain of interest. It is called system because it consists of both problem solving component and a support component.

- The process of building Expert System is called **knowledge engineering** and is done by **knowledge Engineer**

Expert Systems: Introduction

- **Knowledge Engineer**
  - is a human with a background of computer science and AI who knows how to built an Expert System
  - decides how to represent the knowledge and helps programmer to write codes
  - creates a relation with human expert to elicit knowledge from him/her
- **Knowledge Engineering**
  - is the acquisition of knowledge from human expert or other sources
- **Expert**
  - evaluates the expert system and gives report to knowledge engineer
  - the process continues until the performance is found satisfactory by the experts

Expert system

Expert systems provide the following important features:

- Facility for non-expert personnel to solve problems that require some expertise
- Speedy solution
- Reliable solution
- Cost reduction
- Power to manage without human experts
- Wider areas of knowledge

Use of expert systems is specially recommended when:

- Human experts are difficult to find
- Human experts are expensive
- Knowledge improvement is important
- The available information is poor, partial, incomplete
- Problems are incompletely defined
- There is lack of knowledge among all those who need it
- The problem is rapidly changing legal rules and codes

Expert Systems: Characteristics

- High Performance:
  - The quality of the advice given to the system should be very high
  - The system must respond at the competency equal to or better than human expert
- Expertise
  - Real expert → produce good solutions + fast
  - Must be skillful to produce efficient + effective solution eliminating wasteful or unnecessary calculations
  - Must be robust
- Adequate response time
  - Should respond within reasonable amount of time comparable to or better than time taken by human experts to reach the decision
  - For real time-system time, constraint specified on the performance is severe when response required is within certain time interval

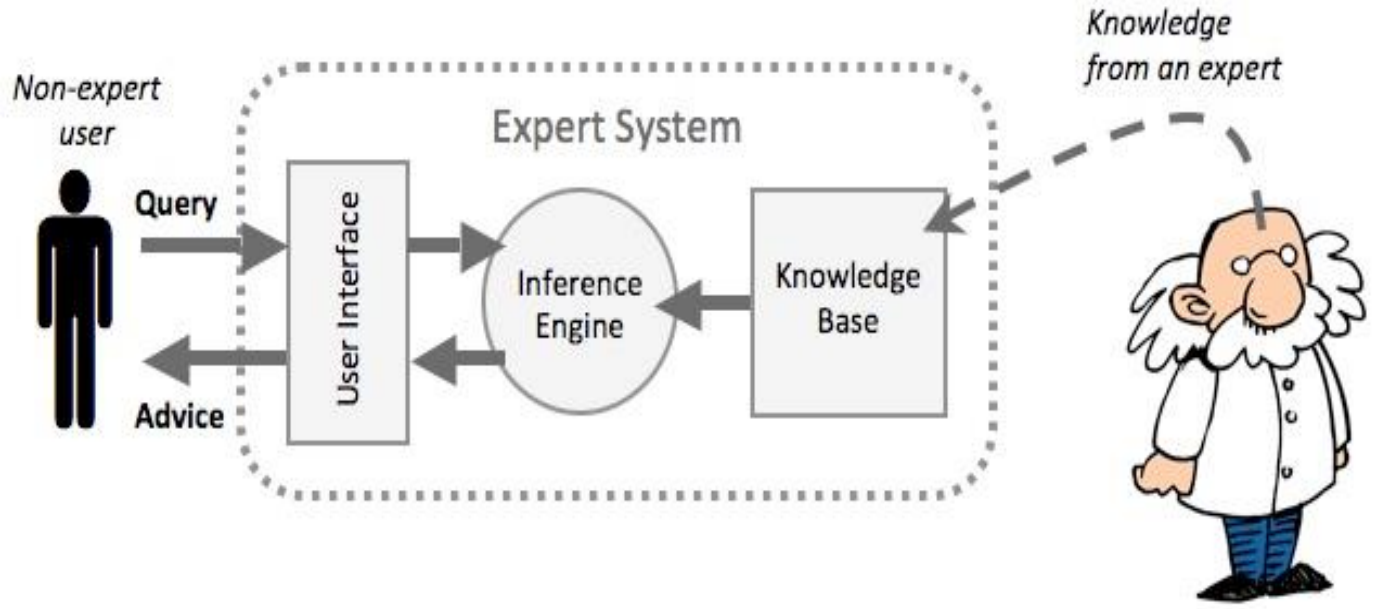
Expert Systems: Characteristics

- Good reliability
  - Must be reliable as that of human experts
- Self knowledge
  - Explanation facility is knowledge that explains how the system arrives its answer
  - The ability to examine their reasoning process and explain their operation is one of the innovative and important quality of expert system
  - Self-knowledge is important in expert system because:
    - User tends to have more faith in results and more confidence in system
    - System development is faster since the system is very easy to compile and debug
    - Assumptions underlying the system's operation is explicit rather than implicit
    - Effect of a change on the system operation is easily tested and predicted

Expert Systems: Characteristics

- Understandable
  - System should be able to explain the steps of reasoning while executing
  - Should have explanation capability similar to the reasoning ability of human experts
- Justification
  - Should justify the solution thus proposed
  - Justification is done via reasoning
  - Ex: if the system is rule based then it provides all the rules and facts that have been used to achieve the answer
- Flexibility
  - Huge knowledge → must have efficient mechanism to add, change and delete knowledge

Architecture of an Expert System



Architecture of an Expert System

- **Knowledge Base**
  - It contains domain-specific and high-quality knowledge. Knowledge is required to exhibit intelligence. The success of any ES majorly depends upon the collection of highly accurate and precise knowledge.
  - What is Knowledge?
    - The data is collection of facts. The information is organized as data and facts about the task domain. **Data, information,** and **past experience** combined together are termed as knowledge.
  - Components of Knowledge Base
    - The knowledge base of an ES is a store of both, factual and heuristic knowledge.
    - **Factual Knowledge** − It is the information widely accepted by the Knowledge Engineers and scholars in the task domain.
    - **Heuristic Knowledge** − It is about practice, accurate judgment, one's ability of evaluation, and guessing.

Architecture of an Expert System

- **Inference Engine**
  - Use of efficient procedures and rules by the Inference Engine is essential in deducting a correct, flawless solution.
  - In case of knowledge-based ES:
    - the Inference Engine acquires and manipulates the knowledge from the knowledge base to arrive at a particular solution.
  - In case of rule based ES, the inference engine:
    - Applies rules repeatedly to the facts, which are obtained from earlier rule application.
    - Adds new knowledge into the knowledge base if required.
    - Resolves rules conflict when multiple rules are applicable to a particular case.

Architecture of an Expert System

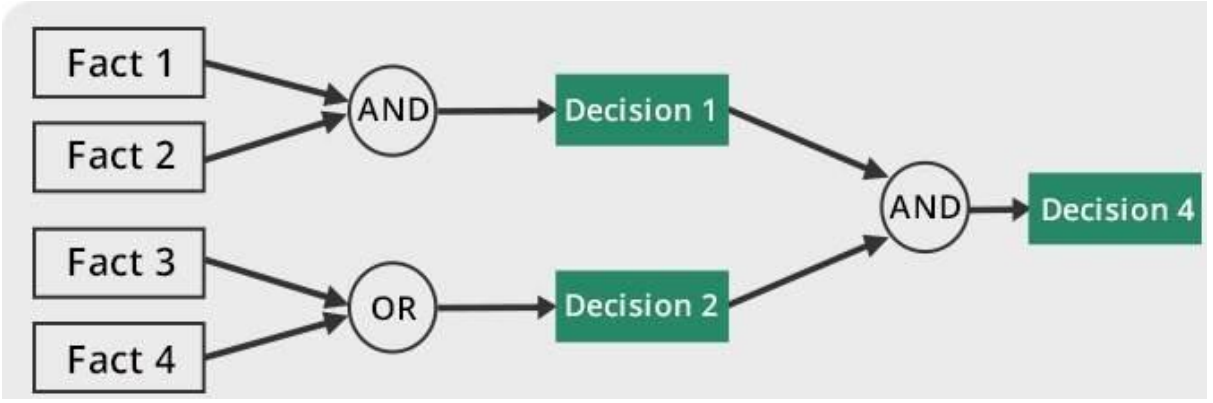
- **User Interface**
  - User interface provides interaction between user of the ES and the ES itself. It is generally Natural Language Processing so as to be used by the user who is well-versed in the task domain. The user of the ES need not be necessarily an expert in Artificial Intelligence.
  - It explains how the ES has arrived at a particular recommendation. The explanation may appear in the following forms −
    - Natural language displayed on screen.
    - Verbal narrations in natural language.
    - Listing of rule numbers displayed on the screen.
  - The user interface makes it easy to trace the credibility of the deductions.

More on Inference Engine

- To recommend a solution, the Inference Engine uses the following strategies :
  - Forward Chaining
  - Backward Chaining

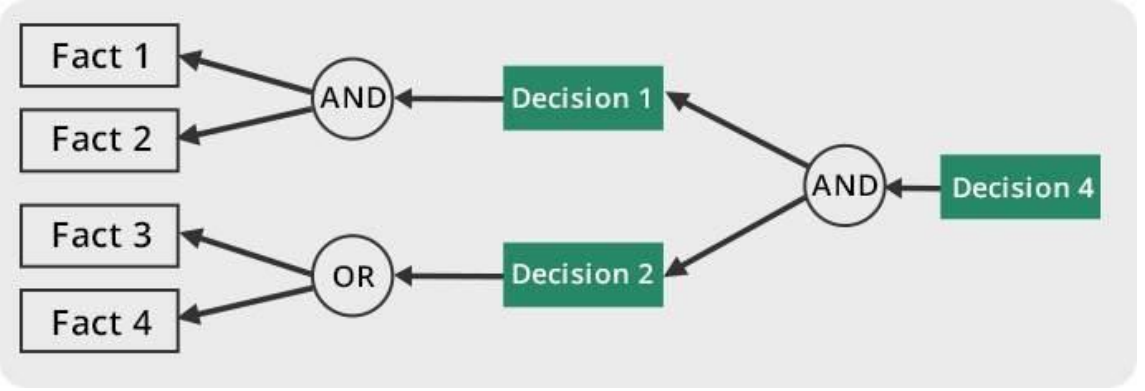
Forward Chaining

- It is a strategy of an expert system to answer the question, **“What can happen next?”**
- Here, the Inference Engine follows the chain of conditions and derivations and finally deduces the outcome. It considers all the facts and rules, and sorts them before concluding to a solution.
- This strategy is followed for working on conclusion, result, or effect. For example, prediction of share market status as an effect of changes in interest rates.



Backward Chaining

- With this strategy, an expert system finds out the answer to the question, **“Why this happened?”**
- On the basis of what has already happened, the Inference Engine tries to find out which conditions could have happened in the past for this result. This strategy is followed for finding out cause or reason. For example, diagnosis of blood cancer in humans.



Development of Expert System

- An expert system typically is developed and refined over a period of several years.
- We can divide the process of expert system development into five distinct stages.
- In practice, it may not be possible to break down the expert system development cycle precisely.
- However, an examination of these six stages may serve to provide us with some insight into the ways in which expert systems are developed.

Development of Expert System

The process of ES development is iterative. Steps in developing the ES include −

- **Identify Problem Domain**
  - The problem must be suitable for an expert system to solve it.
  - Find the experts in task domain for the ES project.
  - Establish cost-effectiveness of the system.
- **Design the System**
  - Identify the ES Technology
  - Know and establish the degree of integration with the other systems and databases.
  - Realize how the concepts can represent the domain knowledge best.
- **Develop the Prototype**

  From Knowledge Base: The knowledge engineer works to −
  - Acquire domain knowledge from the expert.
  - Represent it in the form of If-THEN-ELSE rules.

Development of Expert System

- **Test and Refine the Prototype**
  - The knowledge engineer uses sample cases to test the prototype for any deficiencies in performance.
  - End users test the prototypes of the ES.
- **Develop and Complete the ES**
  - Test and ensure the interaction of the ES with all elements of its environment, including end users, databases, and other information systems.
  - Document the ES project well.
  - Train the user to use ES.
- **Maintain the ES**
  - Keep the knowledge base up-to-date by regular review and update.
  - Cater for new interfaces with other information systems, as those systems evolve.

Human Expert Vs Expert System

Factor	Human Expert	Expert System
Time (can be obtained)	Working days only	Anytime
Geography	Local	Anywhere
Safety	Cannot be replaced	Can be replaced
Damages	Yes	No
Speed and Efficiency	Changes	Consistent
Cost	High	Intermediate

Expert Systems Limitations

- Limitations of the technology
- Difficult knowledge acquisition
- ES are difficult to maintain
- High development costs

Applications of Expert System

Application	Description
Design Domain	Camera lens design, automobile design.
Medical Domain	Diagnosis Systems to deduce cause of disease from observed data, conduction medical operations on humans.
Monitoring Systems	Comparing data continuously with observed system or with prescribed behavior such as leakage monitoring in long petroleum pipeline.
Process Control Systems	Controlling a physical process based on monitoring.
Knowledge Domain	Finding out faults in vehicles, computers.
Finance/Commerce	Detection of possible fraud, suspicious transactions, stock market trading, Airline scheduling, cargo scheduling.

Stages of Expert System Development:

- An expert system typically is developed and refined over a period of several years.
- We can divide the process of expert system development into five distinct stages. In practice, it may not be possible to break down the expert system development cycle precisely.
- However, an examination of these five stages may serve to provide us with some insight into the ways in which expert systems are developed.

Benefits of Expert Systems

- **Availability** − They are easily available due to mass production of software.
- **Speed** − They offer great speed. They reduce the amount of work an individual puts in.
- **Less Error Rate** − Error rate is low as compared to human errors.
- **Reducing Risk** − They can work in the environment dangerous to humans.
- **Steady response** − They work steadily without getting motional, tensed or fatigued.

Natural Language Processing

Natural Language

- In philosophy of language, a natural language or ordinary language is any language that has evolved naturally in humans through use and repetition without conscious planning or premeditation.
- Natural languages can take different forms, such as speech or signing.
- They are distinguished from constructed and formal languages such as those used to program computers or to study logic.
- In computing, natural language refers to a human language such as English, Russian, German, or Japanese as distinct from the typically artificial command or programming language with which one usually talks to a computer.

- Humans perceive and communicate through their five basic senses of sight, hearing, touch, smell and taste, and their ability to generate meaningful utterances.
- Developing programs that understand a natural language is a difficult problem.

- Developing programs to understand natural language is important in AI because a natural form of communication with systems is essential.
- AI programs must be able to communicate with their human counterparts in a natural way, and natural language is one of the most important mediums for that purpose.
- So, Natural Language Processing (NLP) is the field that deals with the computer processing of natural languages, mainly evolved by people working in the field of Artificial Intelligence.

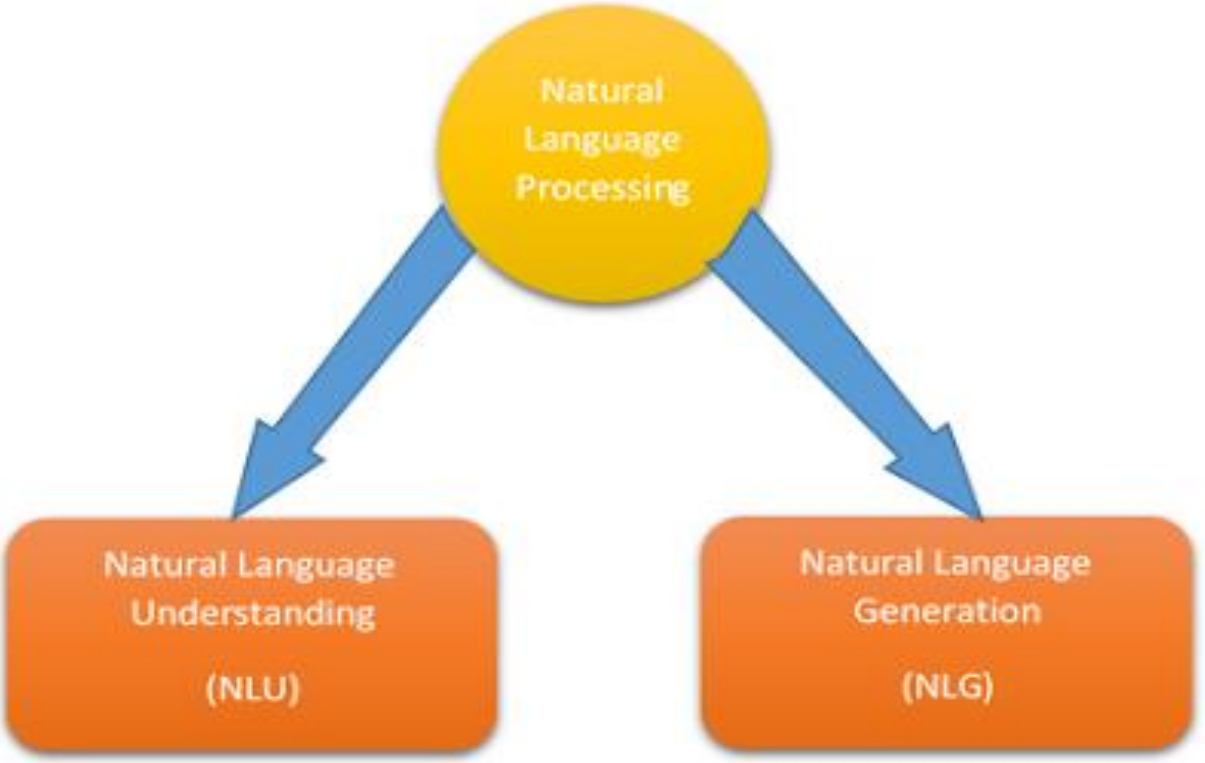
What makes NLP challenging?

Natural Language contain infinity of different sentences. No matter how many sentences a person has heard or seen, new ones can always be produced. Also, there is much ambiguity in a natural language. Many words have several meanings and sentences can have different meanings in different contexts. This makes the creation of programs that understand a natural language, one of the most challenging tasks in AI.

Some of the better-known applications of NLP include:

Voice recognition software: Translates speech into input for word processors or other applications;
Text-to-speech synthesizers: Read text aloud for users such as the hearing-impaired;
Grammar and style checkers: Analyze text in an attempt to highlight errors of grammar or usage;
Machine translation: Systems which automatically render a document such as a web page in another language.

Components in NLP



Natural Language Generation

- NLG also referred to as text generation, is a subfield of natural language processing.
- Natural Language Generation (NLG) is the natural language processing task of generating natural language from a machine representation system such as a knowledge base or a logical form.
- NLG system is like a translator that converts a computer based representation into a natural language representation.

NLG includes

Text planning : - Retrieving the relevant content from database. Here database can be includes vocabulary, sentences, knowledge, sample data and many more.

Sentence planning : - We get our content using text planning now next step to do is choosing required words and forming meaningful sentence setting the words in right grammatical way.

Text realization :- We have all the thing need to create actual text in humans language.

Natural Language Understanding

- It deals with machine reading comprehension : ability to read text, to do process on it, and understand its meaning.
- It is involved with mapping the given inputs in useful representations and analyzing the different aspects of the language.
- Many words have several meaning such as can, bear, fly, bank etc, and sentences have different meanings in different contexts.
- Understanding the language is not only the transmission of words. It also requires inference about the speakers' goal, knowledge as well as the context of the interaction

Difficulties in NLU

- NL has an extremely rich form and structure.
- It is very ambiguous.
- Different levels of ambiguity -
  - **Lexical ambiguity** − It is at very primitive level such as word-level.
  - **Syntax Level ambiguity** − A sentence can be parsed in different ways and difficult to figure out the parsing
  - **Referential ambiguity** − A Single reference points to multiple meanings.
    - One input can mean different meanings.
    - Many inputs can mean the same thing.

Levels of knowledge used in Language Understanding

- A language understanding knowledge must have considerable knowledge about the structures of the language including what the words are and how they combine into phrases and sentences.
- It must also know the meanings of the words and how they contribute to the meanings of the sentence and to the context within which they are being used.

The component forms of knowledge needed for an understanding of natural languages are sometimes classified according to the following levels.

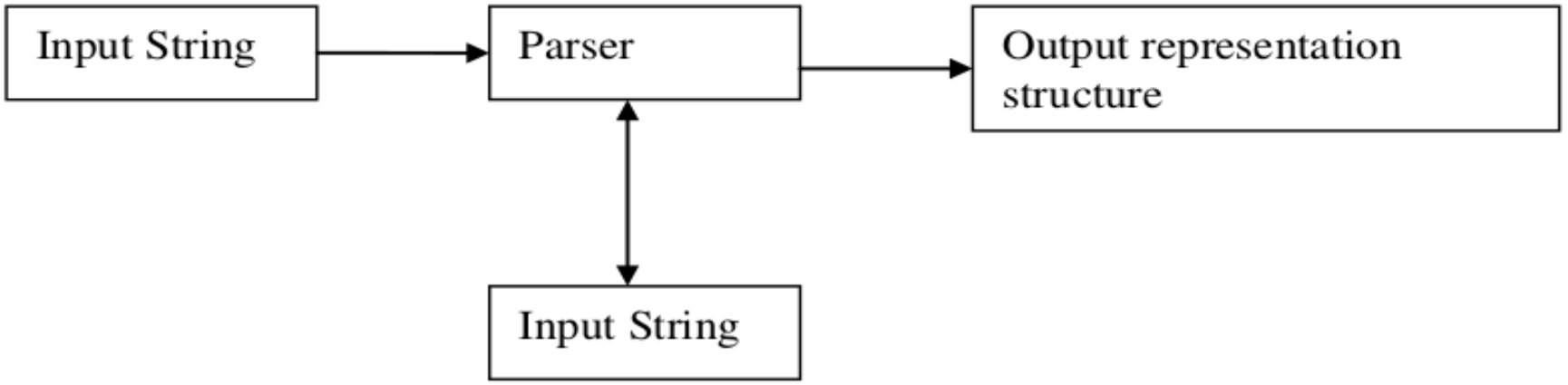
- **Phonological**:
  Relates sound to the words we recognize. A phoneme is the smallest unit of the sound.
- **Morphological:**
  This is lexical (Linguistic unit) knowledge which relates to the word construction from basic units called morphemes. A morpheme is the smallest unit of meaning.
  Eg:- friend + ly = friendly.

- **Syntactic:**
  This knowledge relates to how words are put together or structured together to form grammatically correct sentences in the language.
- **Semantic :**
  This knowledge is concerned with the meanings of words and phrases and how they combine to form sentence meaning
- **Pragmatic:**
  This is high – level knowledge which relates to the use of sentences in different contexts and how the context affects the meaning of the Sentence.
- **World :**
  Includes the knowledge of the physical world, the world of human social interaction, and the roles of goals and intentions in communication.

Basic Parsing Techniques

- Before the meaning of a sentence can be determined, the meanings of its constituent parts must be established.
- This requires knowledge of the structure of the sentence, the meaning of the individual words and how the words modify each other.
- The process of determining the syntactical structure of a sentence is known as parsing

- Parsing is the process of analyzing a sentence by taking it apart word – by – word and determining its structure from its constituent parts and sub parts.
- The structure of a sentence can be represented with a syntactic tree. When given an input string, the lexical parts or terms (root words), must first be identified by type and then the role they play in a sentence must be determined.
- These parts can be combined successively into larger units until a complete tree has been computed.


Input String
Parser
Output representation structure
Input String

A. Top-Down parsing:

- Using Top-Down technique, parser searches for a parse tree by trying to build from the root node S down to the leaves.
- The algorithm starts by assuming the input can be derived by the designated start symbol S.
- The next step is to find the tops of all the trees which can start with S, by looking on the grammar rules with S on left hand side, all the possible trees are generated.
- Top-Down parsing is goal directed search technique.

B. Bottom – Up

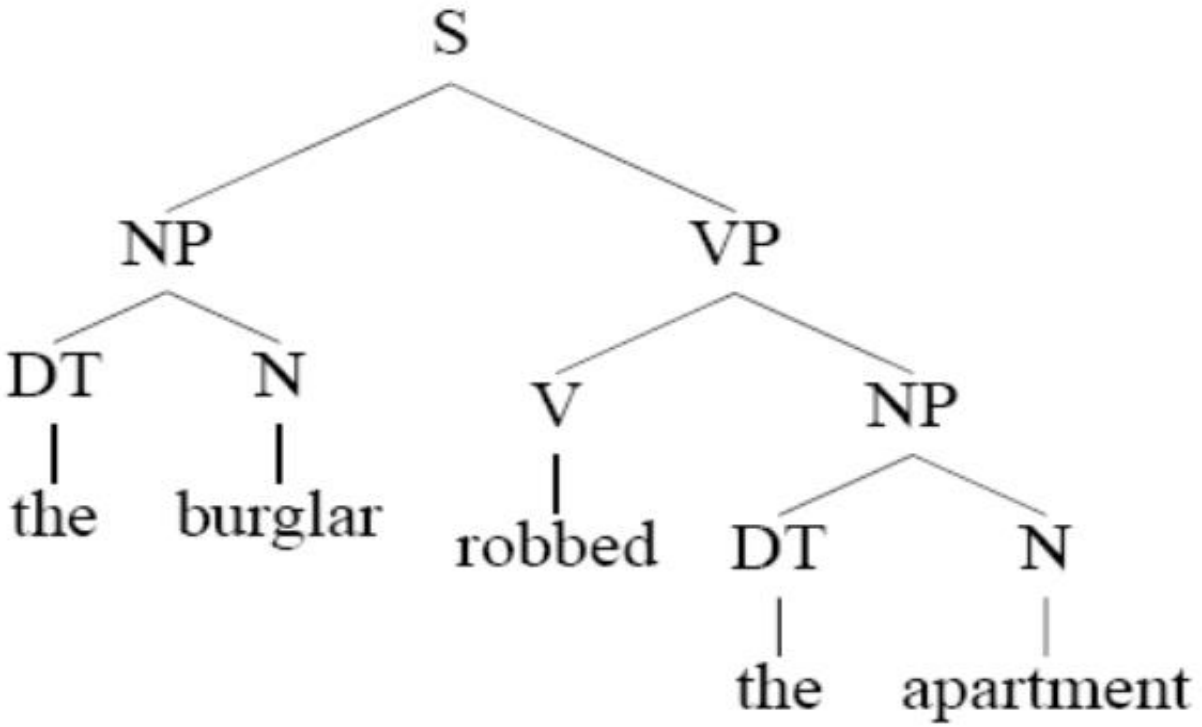
- Bottom – Up parsing starts with the words of input and tries to build trees from the words up, again by applying rules from the grammar one at a time.
- The parse is successful if the parse succeeds in building a tree rooted in the start symbol S that covers all of the input.
- Bottom up parsing is a data directed search.

N = NOUN

V = VERB

DT =

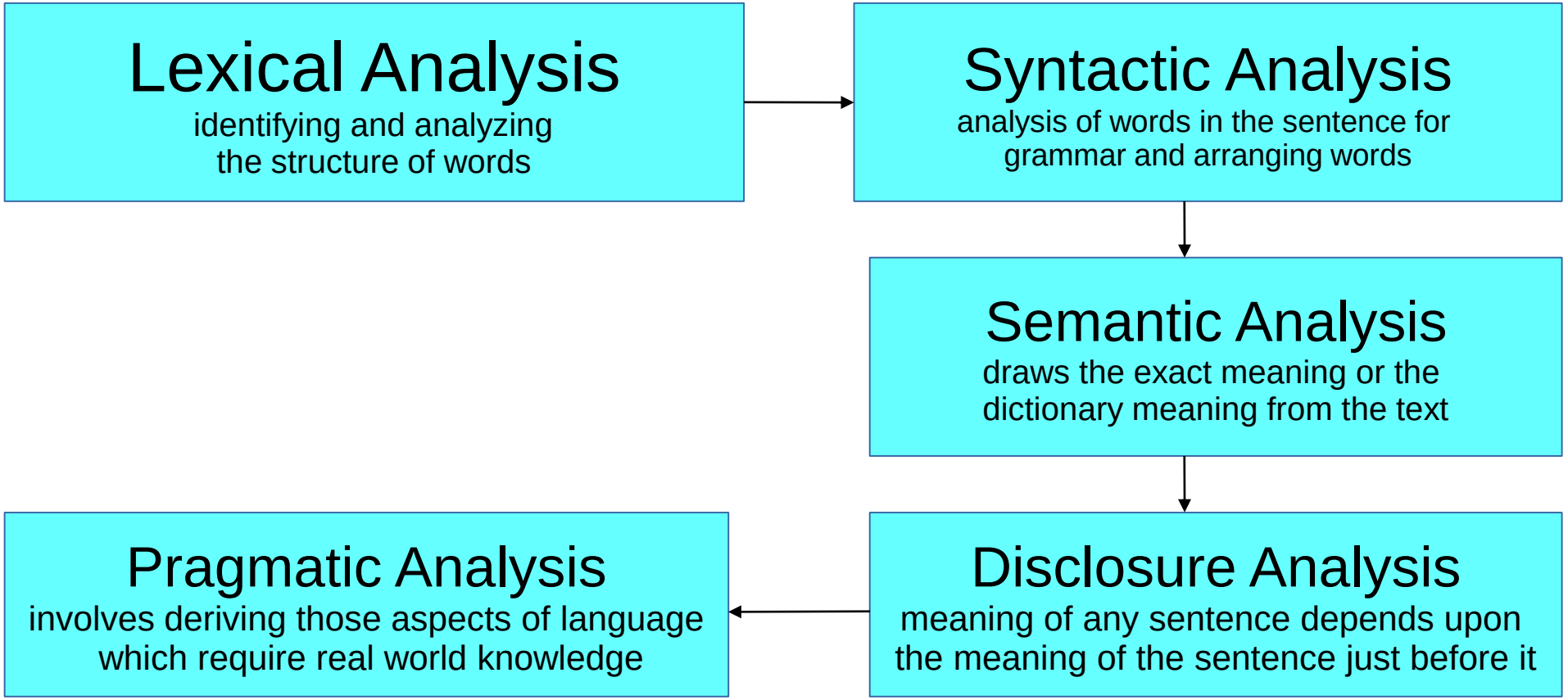
DETERMINER



Assignment

Explain context free grammar

Steps used in NLP



Advantages of NLP

- Relieves burden of Learning syntax
- No training needed for learning Languages

Disadvantages of NLP

- Requires clarification of dialogue
- May not show context
- Unpredictable

Parameters in Natural Language Processing:

- To be exact, the parameters always (regardless of the research field) depend on the task.
- Different parameters may include
  - Auditory Inputs
  - Segmentation
  - Syntax Structure
  - Semantic Structure
  - Pragmatic Analysis

a. Auditory Inputs

- Three of our five senses – sight, hearing and touch – are used as major inputs.
- These are usually referred to as the visual, auditory and tactile inputs respectively.
- They are sometimes called input channels.
- In some cases, an audio output device can be used as an input device, in order to capture produced sound.
    - Microphone
    - MIDI keyboard or other digital musical instrument

b. Text segmentation

- Segmentation: Determining the positions at which topics change in a stream of text or Speech
- Text segmentation is the process of dividing written text into meaningful units, such as words, sentences, or topics.
- The term applies both to mental processes used by humans when reading text, and to artificial processes implemented in computers, which are the subject of natural language processing.

Problems in Text Segmentation

- **Word segmentation** is the problem of dividing a string of written language into its component words.

  For example, ice box = ice-box = icebox

- **Intent segmentation** is the problem of dividing written words into key phrases

  [All things are made of *atoms*]

- **Sentence segmentation** is the problem of dividing a string of written language into its component sentences

- **Topic Segmentation** consists of two main tasks:

  - topic identification
  - text segmentation

c. Syntax Structure

- Syntactic analysis takes an input sentence and produces a representation of its grammatical structure.
- A grammar describes the valid parts of speech of a language and how to combine them into phrases.
- The English Grammar is nearly context free.
- A computer Grammar specifies which sentences are in a language and their parse trees. A parse tree is a hierarchical structure that shows how the grammar applies to the input. Each level of the tree corresponds to the application of one grammar rule.

Consider the sentence

“John saw Mary with a telescope”

Two different readings based on the groupings.

- John saw (Mary with a telescope).
- John (saw Mary with a telescope).

d. Semantic Structure

- Semantic analysis is a process of converting the syntactic representations into meaning representation.
- This involves the following tasks:
    - Word sense determination
    - Sentence level analysis
    - Knowledge representation

- **Word sense**

  Words have different meanings in different contexts.

  Example :

  - Mary had a bat in her office.

    bat = `a baseball thing'

    bat = `a flying mammal'

- **Sentence level analysis**

Once the words are understood, the sentence must be assigned some meaning

- I saw an astronomer with a telescope.

- **Knowledge Representation**

Understanding language requires lots of knowledge.

e. Pragmatic Analysis

- Pragmatics is a subfield of linguistics that studies the ways in which context contributes to meaning
- It is the study of the ways in which language is used and its effect on the listener
- Pragmatic analysis refers to a set of linguistic and logical tools with which analysts develop systematic accounts of logical interactions.

Example

“When is the next flight to Sydney?”
“Does *it* have any seat left?”

Here, “it”, refers to a particular flight to Sydney, not Sydney itself.

Pragmatics – Ambiguity in Language

"I saw an astronomer with a telescope."

Above sentence has a prepositional phrase "with a telescope" which may be attached with either with verb to make phrase "saw something with telescope" or to object noun phrase to make phrase "a astronomer with a telescope". If we do first, then it can be interpreted as "I saw an astronomer who is having a telescope", and if we do second, it can be interpreted as "Using a telescope I saw an astronomer"

Now, to remove such ambiguity, one possible idea is that we have to consider the context. If the knowledge base (KB) can prove that whether the telescope is with astronomer or not, then the problem is solved.

If A says the same sentence “I saw an astronomer with a telescope.” To B, then in practical, it is more probable that, B (listener) realizes that “A has seen astronomer who is having a telescope”. It is because, normally, the word “telescope” belongs to “astronomer”, so it is obvious that B realizes so.

If A has says that “I saw a lady with a telescope.” In this case, B realizes that “A has seen the lady using a telescope”, because the word “telescope” has not any practical relationship with “lady” like “astronomer”.

Neural Networks

A neuron is a cell in brain whose principle function is the collection, Processing, and dissemination of electrical signals.

Brains Information processing capacity comes from networks of such neurons.

Due to this reason some earliest AI work aimed to create such artificial networks/ connectionism / Parallel distributed processing/ Neural computing

Neural Network

- An Artificial Neural Network (ANN) is an information processing paradigm that is inspired by the way biological nervous systems, such as the brain, process information.
- It is composed of a large number of highly interconnected processing elements (neurones) working to solve specific problems.

- ANNs, like people, learn by example.
- An ANN is configured for a specific application, such as pattern recognition or data classification, through a learning process.

- A neural network is a system of hardware and/or software patterned after the operation of neurons in the human brain.
- *A neural network usually involves a large number of processors operating in parallel and arranged in tiers.*
- *The first tier receives the raw input information -- analogous to optic nerves in human visual processing.*
- *Each successive tier receives the output from the tier preceding it, rather than from the raw input -- in the same way neurons further from the optic nerve receive signals from those closer to it.*
- *The last tier produces the output of the system.*

Misconceptions

- Neural Networks are model of human brain
- Bigger size neural Networks works better
- Many Training data exists in neural networks

Why Neural Networks

- **Pattern Extraction :** can be used to extract patterns and detect trends that are too complex to be noticed by either humans or other computer techniques.
- **Expert Systems:** A trained neural network can be thought of as an expert in the category of information it has been given to analyze.
- **Adaptive learning:** An ability to learn how to do tasks based on the data given for training or initial experience.
- **Fault Tolerance via Redundant Information Coding:** Partial destruction of a network leads to the corresponding degradation of performance. However, some network capabilities may be retained even with major network damage

How Do Neural Networks Differ From Conventional Computing?

- ANNs are not sequential.
- There are no complex central processors, rather there are many simple ones which generally do nothing more than take the weighted sum of their inputs from other processors.
- ANNs do not execute programed instructions; they respond in parallel (either simulated or actual) to the pattern of inputs presented to it.
- There are also no separate memory addresses for storing data. Instead, information is contained in the overall activation 'state' of the network.

Units of Neural Network

- **Nodes (units)**

Nodes represent a cell of neural network.

- **Links**

Links are directed arrows that show propagation of information from one node to another node.

- **Activation**

Activation are inputs to or outputs from a unit.

- **Weight**

Each link has weight associated with it which determines strength and sign of the connection.

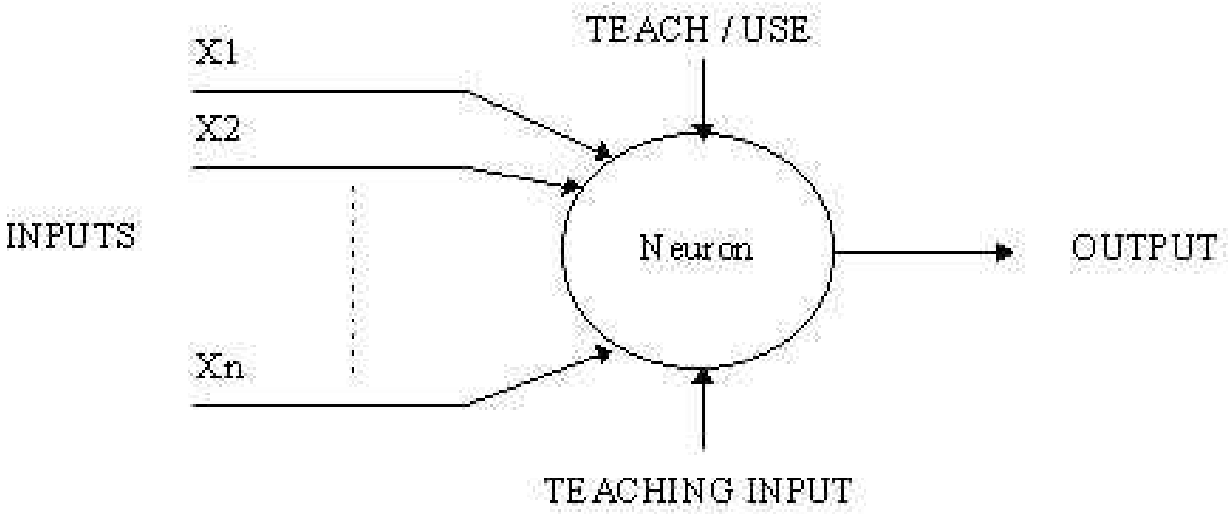
- **Activation function:**

A function which is used to derive output activation from the input activations to a given node is called activation function.

- **Bias:**

Bias is a feature of a statistical technique or of its results whereby the expected value of the results differs from the true underlying quantitative parameter being estimated.

A simple neuron



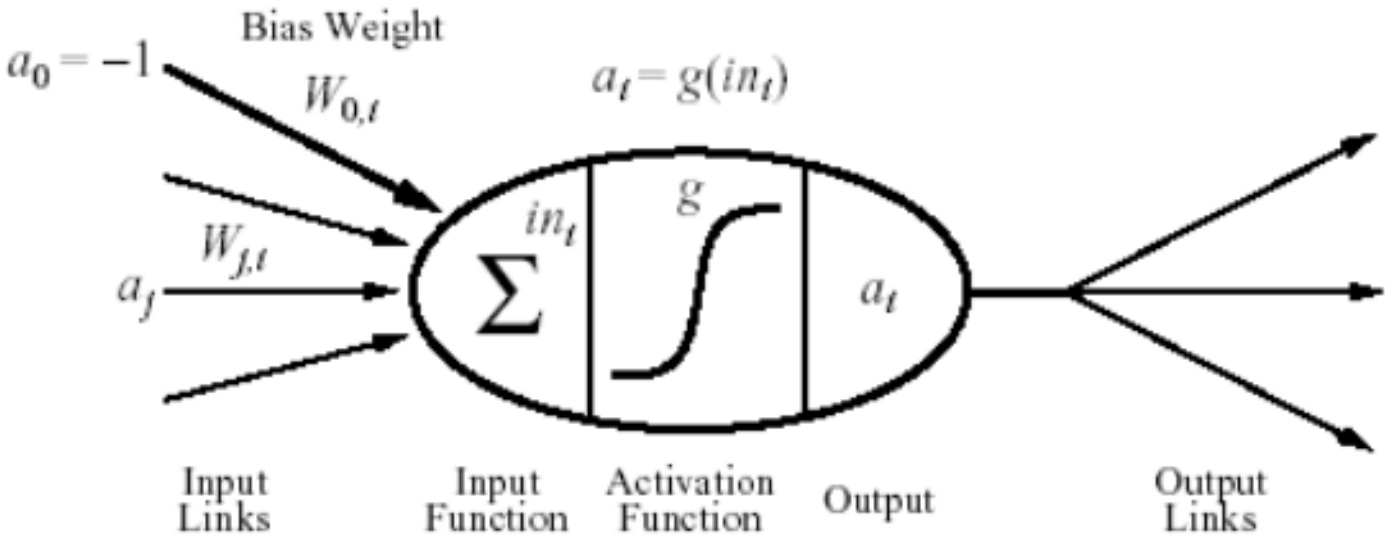
An artificial neuron is a device with many inputs and one output. The neuron has two modes of operation; the training mode and the using mode. In the training mode, the neuron can be trained to fire (or not), for particular input patterns. In the using mode, when a taught input pattern is detected at the input, its associated output becomes the current output.

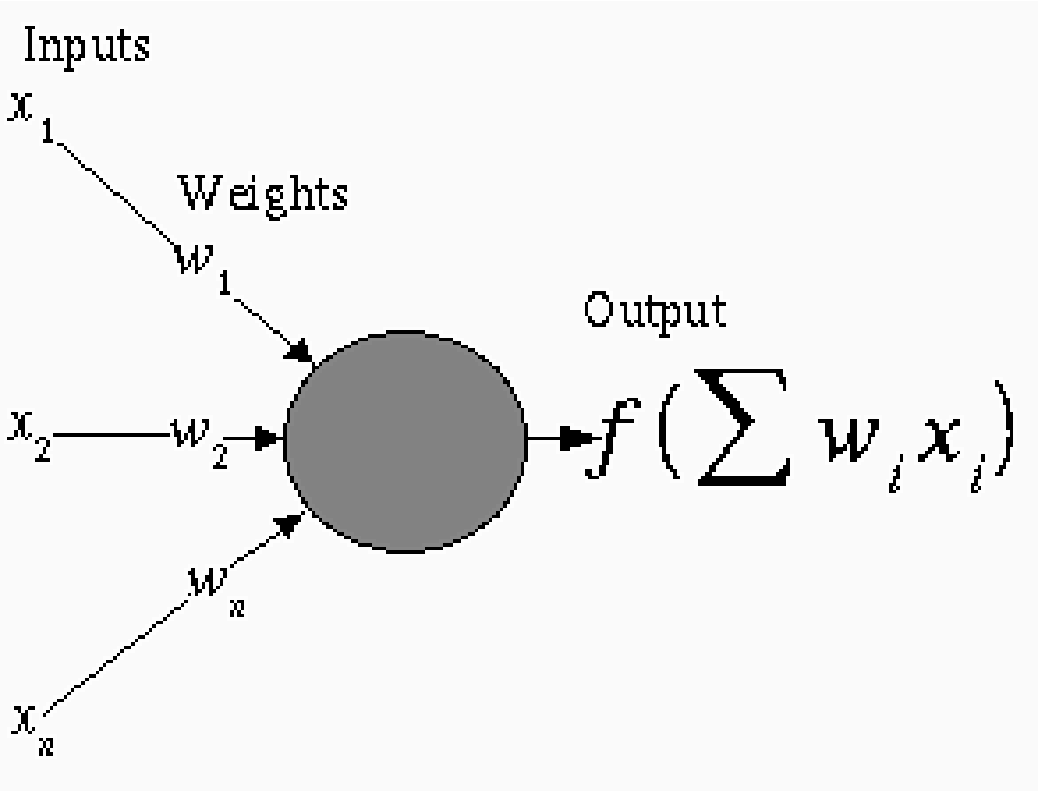
If the input pattern does not belong in the taught list of input patterns, the firing rule is used to determine whether to fire or not.

Simple Model of Neural Network

A simple mathematical model of neuron is devised by McCulloch and Pit is given in the figure given below:

$$g(in_i) = g\left(\sum W_{ij}\, a_j\right)$$

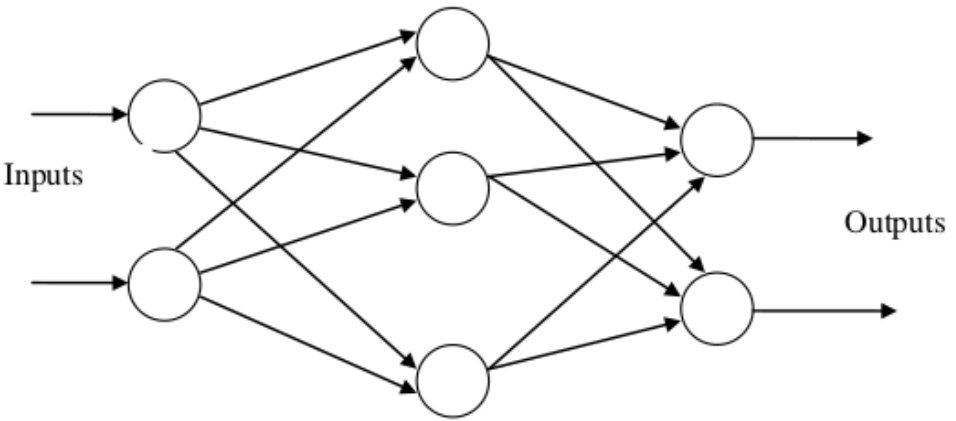



Inputs
x_1
Weights
w_1
Output
x_2
w_2
$f\left(\sum w_i x_i\right)$
w_n
x_n

Architecture of Neural Networks

Feed-forward networks

- Feed-forward ANNs allow signals to travel one way only; from input to output.
- There is no feedback (loops) i.e. the output of any layer does not affect that same layer.
- Feed forward ANNs tend to be straight forward networks that associate inputs with outputs.
- They are extensively used in pattern recognition.
- This type of organization is also referred to as bottom-up or top-down.

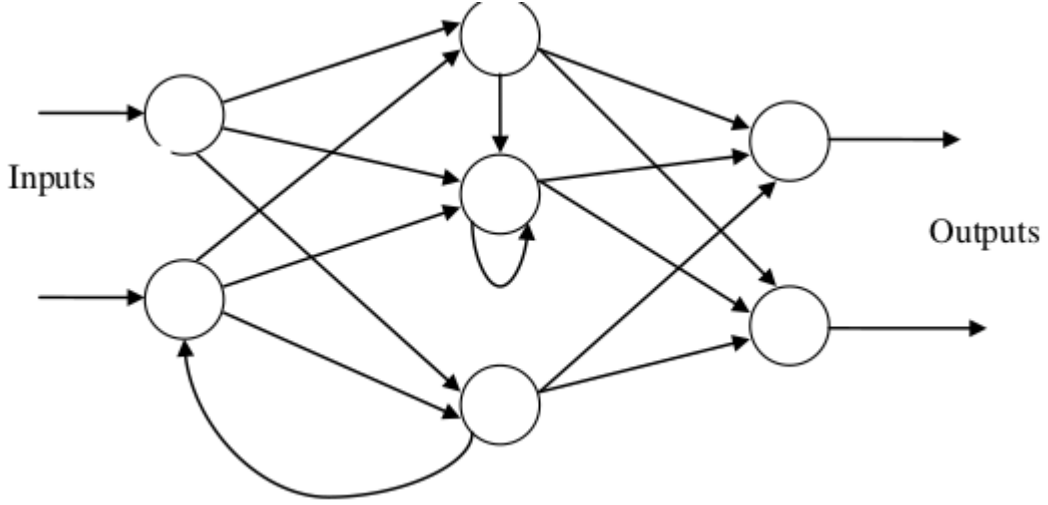


Feedback networks

- Feedback networks can have signals traveling in both directions by introducing loops in the network.

- Feedback networks are very powerful and can get extremely complicated.

- Feedback networks are dynamic; their 'state' is changing continuously until they reach an equilibrium point. They remain at the equilibrium point until the input changes and a new equilibrium needs to be found.



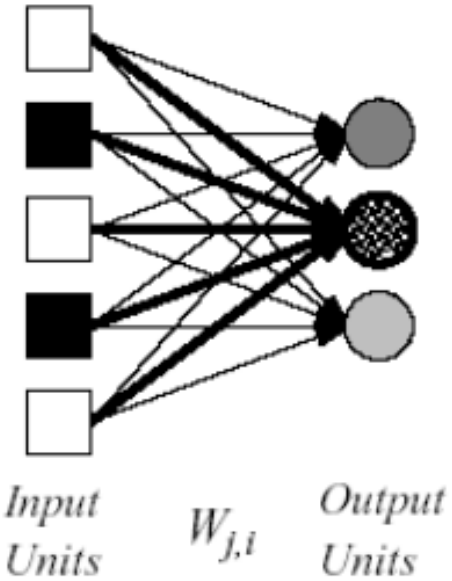
In forward propagation, we apply a set of weights to the input data and calculate an output. For the first forward propagation, the set of weights is selected randomly.

In back propagation, we measure the margin of error of the output and adjust the weights accordingly to decrease the error

Types of Feed Forward Neural Network

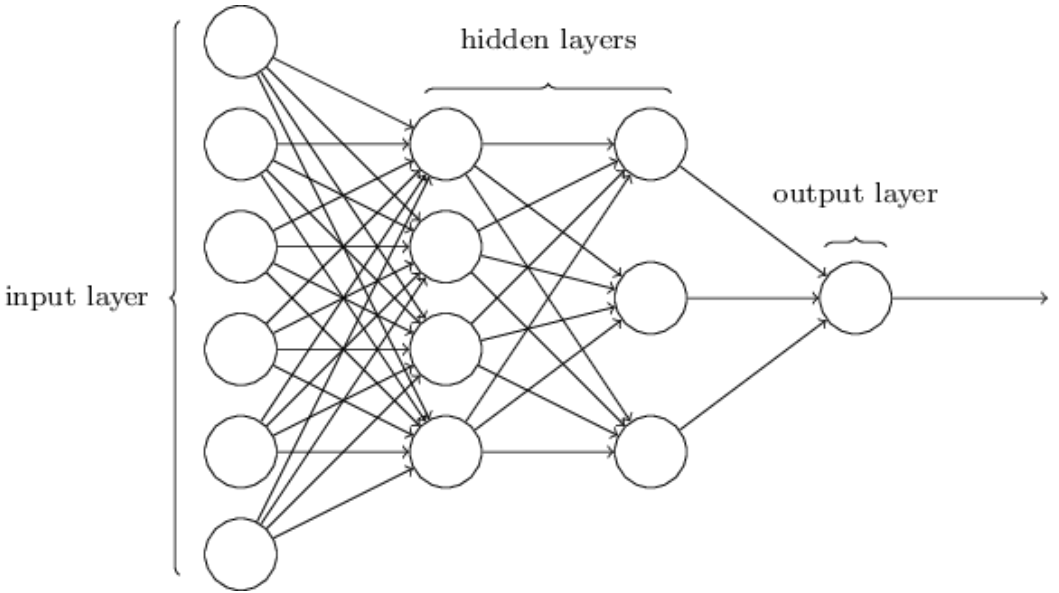
Single-layer neural networks (perceptrons)

- A neural network in which all the inputs connected directly to the outputs is called a single- layer neural network, or a perceptron network.
- Since each output unit is independent of the others each weight affects only one of the outputs.



Multilayer neural networks (perceptrons)

- The neural network which contains input layers, output layers and some hidden layers also is called multilayer neural network.
- The advantage of adding hidden layers is that it enlarges the space of hypothesis. Layers of the network are normally fully connected.



Neural Networks in Practice

Since neural networks are best at identifying patterns or trends in data, they are well suited for prediction or forecasting needs including:

- sales forecasting
- industrial process control
- customer research
- data validation
- risk management
- target marketing

Hebbian Learning:

- Hebb's postulate of learning is :-

  "When an axon of cell A is near enough to excite a cell B and repeatedly taking part in firing it, some growth process or metabolic changes take place in one or both cells such that A's efficiency as one of the cells firing B is increased"

- **The weight between two neurons increases if the two neurons activate simultaneously—and reduces if they activate separately.**

- Nodes that tend to be either both positive or both negative at the same time have strong positive weights, while those that tend to be opposite have strong negative weights.

Hebb's Algorithm:

Step 0: initialize all weights to 0

Step 1: Given a training input, s, with its target output, t, set the activations of the input units: $x_i = s_i$

Step 2: Set the activation of the output unit to the target value: $y = t$

Step 3: Adjust the weights: $w_i(new) = w_i(old) + x_i y$

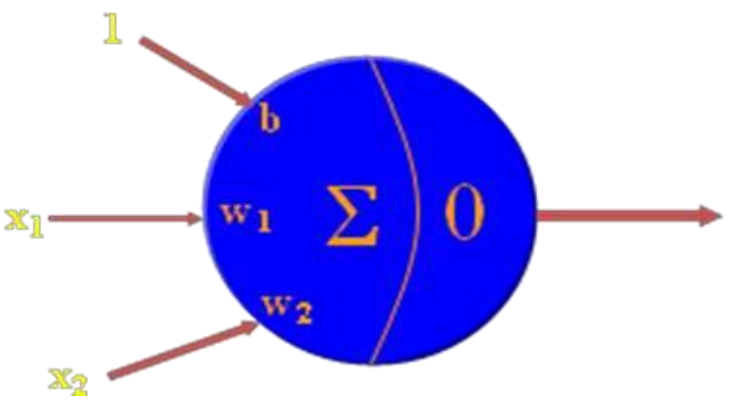
Step 4: Adjust the bias (just like the weights): $b(new) = b(old) + y$

(bias: the difference between the expectation of sample estimator and true value)

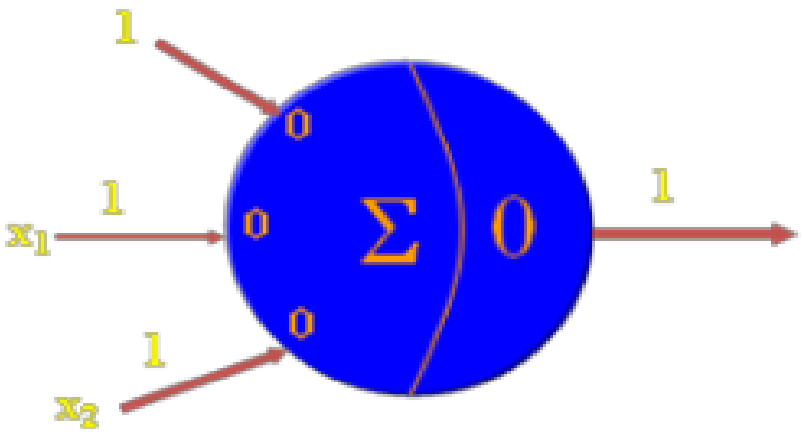
Construct a Hebb Net which performs like an AND function, that is, only when both features are “active” will the data be in the target class.

TRAINING SET (with the bias input always at 1):

x1	x2	bias	Target
1	1	1	1
1	-1	1	-1
-1	1	1	-1
-1	-1	1	-1



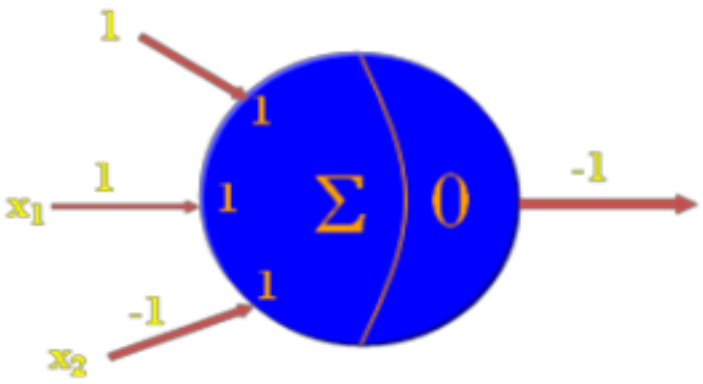
- **Initialize the weights to 0**
- **Present the first input (1 1 1) with target of 1**



Update the weights:

$$w_1(new) = w_1(old) + x_1 t$$
$$= 0 + 1 = 1$$
$$w_2(new) = w_2(old) + x_2 t$$
$$= 0 + 1 = 1$$
$$b(new) = b(old) + t$$
$$= 0 + 1 = 1$$

- **Present the second input (1 -1 1) with target of -1**



Update the weights:

$w_1(new) = w_1(old) + x_1 t$

$= 1 + 1(-1) = 0$

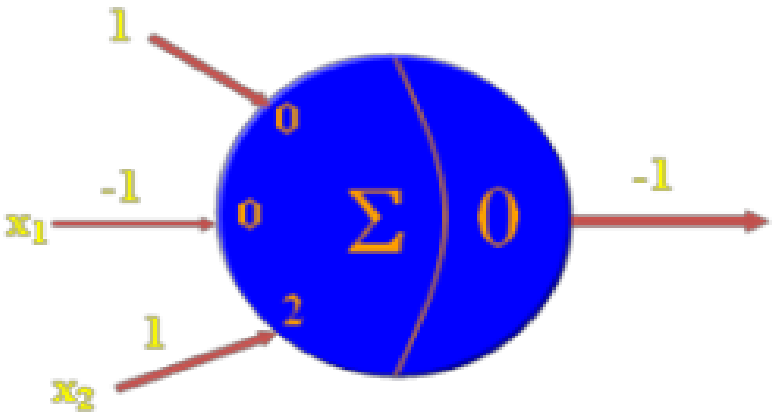
$w_2(new) = w_2(old) + x_2 t$

$= 1 + (-1)(-1) = 2$

$b(new) = b(old) + t$

$= 1 + (-1) = 0$

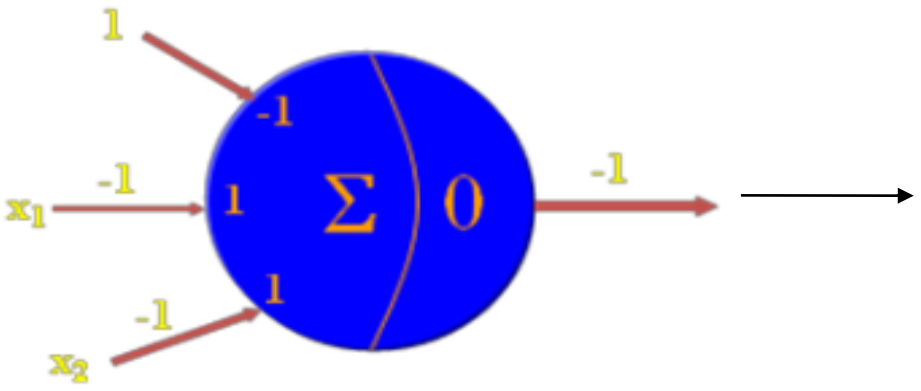
- **Present the third input (-1 1 1) with target of -1**



Update the weights:

$$w_1(new) = w_1(old) + x_1 t$$
$$= 0 + (-1)(-1) = 1$$
$$w_2(new) = w_2(old) + x_2 t$$
$$= 2 + 1(-1) = 1$$
$$b(new) = b(old) + t$$
$$= 0 + (-1) = -1$$

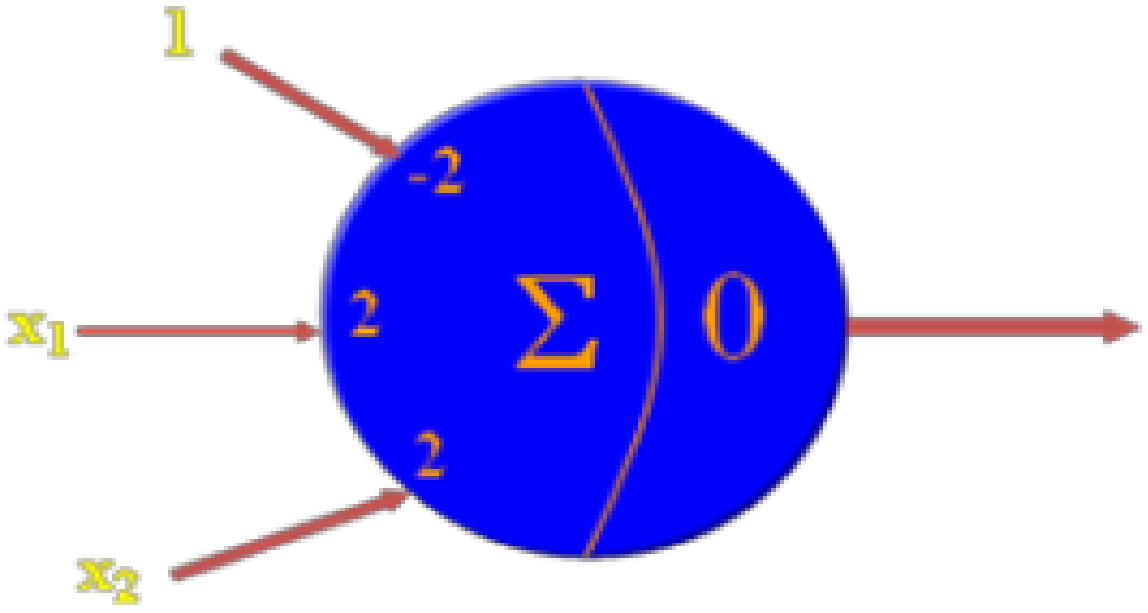
- **Present the Fourth input (-1 -1 1) with target of -1**



Update the weights:

$$w_1(new) = w_1(old) + x_1 t$$
$$= 1 + (-1)(-1) = 2$$
$$w_2(new) = w_2(old) + x_2 t$$
$$= 1 + (-1)(-1) = 2$$
$$b(new) = b(old) + t$$
$$= -1 + (-1) = -2$$

The Final Neuron



Perceptron

- The perceptron is an algorithm for supervised learning
- Functions that can decide whether an input, represented by a vector of numbers, belongs to some specific class or not

Perceptron Learning Theory

- Perceptron (an element of artificial Neural Network consisting of one or more layer of artificial neuron)
- The term "Perceptrons" was coined by Frank RosenBlatt in 1962 and is used to describe the connection of simple neurons into networks.
- These networks are simplified versions of the real nervous system where some properties are exaggerated and others are ignored.

- We need to train the neural network so they can do things that are useful.
- To do this we must allow the neuron to learn from its mistakes.
- There is in fact a learning paradigm that achieves this, it is known as supervised learning and works in the following manner.

1. Set the weight and thresholds of the neuron to random values.
2. Present an input.
3. Calculate the output of the neuron.
4. Alter the weights to reinforce correct decisions and discourage wrong decisions, hence reducing the error. So for the network to learn we shall increase the weights on the active inputs when we want the output to be active, and to decrease them when we want the output to be inactive.
5. Now present the next input and repeat steps 3.

Delta Rule/ Widrow – Hoff Rule

- Called as the Least Mean Square (LMS) method Delta rule is one of the most commonly used learning rules.
- It is a special case of the more general back propagation algorithm.
- For a given input vector, the output vector is compared to the correct answer, If the difference is zero, no learning takes place; otherwise, the weights are adjusted to reduce this difference.

For a neuron j with activation function g(x) , the delta rule for j 's i^{th} weight w_{ji} is given by

$$\Delta w_{ji} = \alpha(t_j - y_j)g'(h_j)x_i$$

where

α is a small constant called *learning rate*

$g(x)$ is the neuron's activation function

t_j is the target output

h_j is the weighted sum of the neuron's inputs

y_j is the actual output

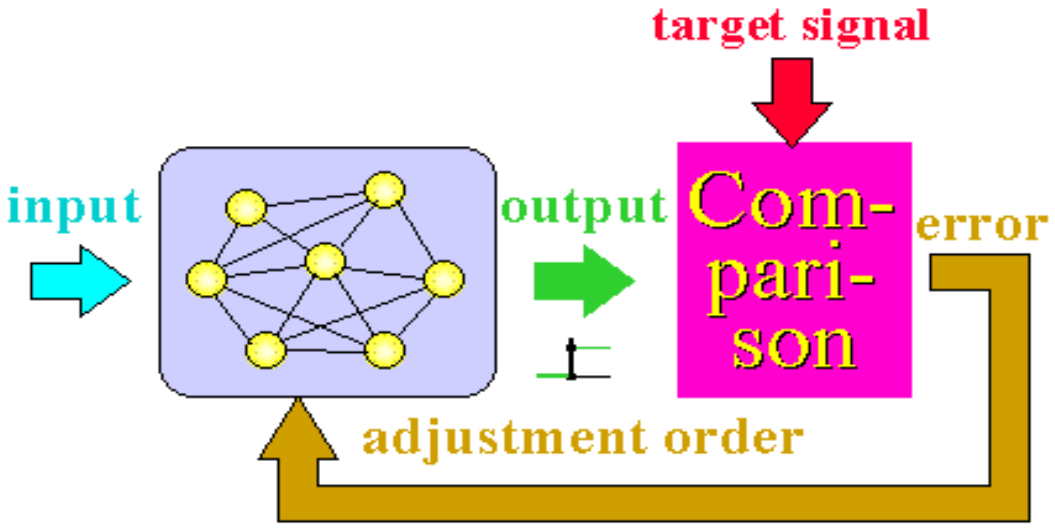
x_i is the ith input.

It holds that $h_j = \sum x_i w_{ji}$ and $y_j = g(h_j)$.

Back propagation

- It is a supervised learning method, and is an implementation of the Delta rule.
- It requires a teacher that knows, or can calculate, the desired output for any given input. It is most useful for feed-forward networks.
- The term is an abbreviation for "backwards propagation of errors".
- Back propagation networks are necessarily multilayer perceptrons (usually with one input, one hidden, and one output layer).

As the algorithm's name implies, the errors (and therefore the learning) propagate backwards from the output nodes to the inner nodes. So technically speaking, back propagation is used to calculate the gradient of the error of the network with respect to the network's modifiable weights.


target signal
input
output
Com-
pari-
son
error
adjustment order

Back propagation technique

- Randomly choose the initial weights
- While error is too large

- For each training pattern (presented in random order)
  - Apply the inputs to the network
  - Calculate the output for every neuron from the input layer, through the hidden layer(s), to the output layer
  - Calculate the error at the outputs
  - Use the output error to compute error signals for pre-output layers
  - Use the error signals to compute weight adjustments
  - Apply the weight adjustments
- Periodically evaluate the network performance

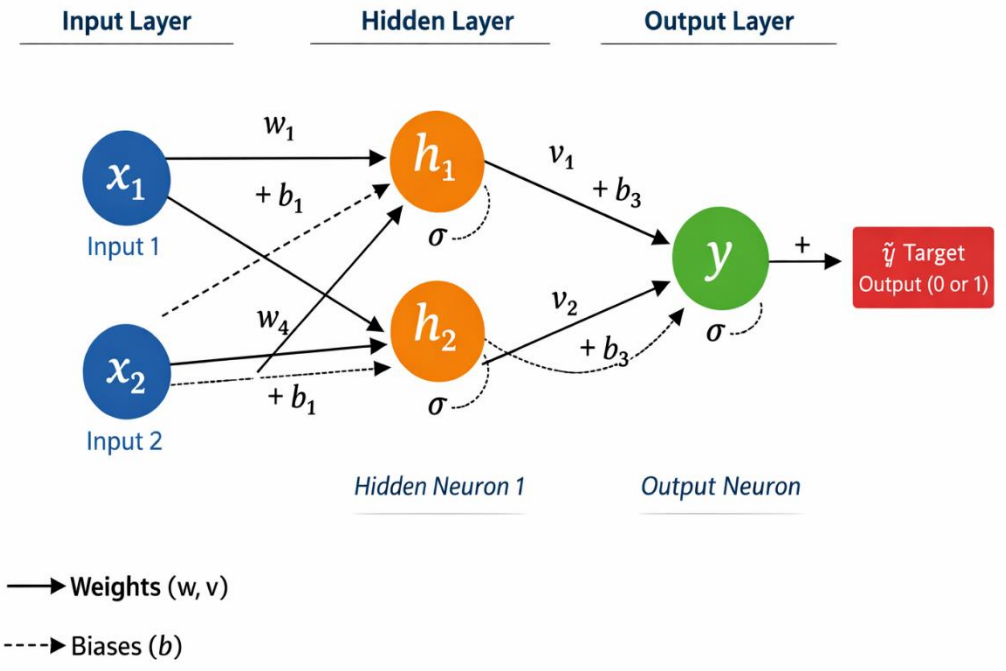
Example

Construct a Neural Network which performs like an XOR function.

- This problem cannot be implemented by single-layer neural network (like a perceptron) , but a **multi-layer neural network (MLP)** can.
- The XOR truth table:

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

- This is not linearly separable, so we need at least one hidden layer.
- Also known as ***Mcculloch Pitts Neuron Model***
- Neural Network Architecture
  - A simple network:
    - ***Input layer: 2 neurons (x_1, x_2)***
    - ***Hidden layer: 2 neurons***
    - ***Output layer: 1 neuron***



Hidden Layer

$$h_1 = \sigma(w_1 x_1 + w_2 x_2 + b_1), \quad h_2 = \sigma(w_3 x_1 + w_4 x_2 + b_2)$$

Output Layer

$$y = \sigma(v_1 h_1 + v_2 h_2 + b_3)$$

σ = activation function (sigmoid function)

$$\sigma(x) = \frac{1}{1+e^{-x}} \qquad \sigma'(x) = \sigma(x)(1 - \sigma(x))$$

First neuron learns ***OR***

Second neuron learns ***AND***

Output computes:

$$XOR = OR - AND$$

Initialize

w1 = 0.1, w2 = 0.2 w3 = 0.3, w4 = 0.4
b1 = 0.1, b2 = 0.1
v1 = 0.5, v2 = 0.6 and b3 = 0.1

Let’s take the first input:

x1 = 1, x2 = 0 and Target(t) = 1

$$z_1 = 0.1(1) + 0.2(0) + 0.1 = 0.2$$

$$h_1 = \sigma(0.2) \approx 0.5498$$

$$z_2 = 0.3(1) + 0.4(0) + 0.1 = 0.4$$

$$h_2 = \sigma(0.4) \approx 0.5987$$

Output Layer:

$$z_3 = 0.5(0.5498) + 0.6(0.5987) + 0.1 \approx 0.7341$$

$$y = \sigma(0.7341) \approx 0.6757$$

Compute Error:

$$E = \frac{1}{2}(t - y)^2$$

$$E \approx \frac{1}{2}(1 - 0.6757)^2 \approx 0.0527$$

Output Layer Error:

$$\delta_{out} = (y - t) \cdot y(1 - y)$$

$$= (0.6757 - 1)(0.6757)(1 - 0.6757)$$

$$= (-0.3243)(0.2192) = -0.0711$$

For *h1*

$$\delta_{h1} = \delta_{out} \cdot v_1 \cdot h_1(1 - h_1)$$

$$= (-0.0711)(0.5)(0.5498 \times 0.4502)$$

$$= (-0.0711)(0.5)(0.2475) = -0.0088$$

For *h2*

$$\delta_{h2} = \delta_{out} \cdot v_2 \cdot h_2(1 - h_2)$$

$$= (-0.0711)(0.6)(0.5987 \times 0.4013)$$

$$= (-0.0711)(0.6)(0.2403) = -0.0102$$

Update Weights

Update Hidden → Output Weights

$$v_1^{new} = v_1 - \eta \cdot \delta_{out} \cdot h_1$$

$$= 0.5 - (0.1 \times -0.0711 \times 0.5498) = 0.5039$$

$$v_2^{new} = 0.6 - (0.1 \times -0.0711 \times 0.5987) = 0.6043$$

$$b_3^{new} = 0.1 - (0.1 \times -0.0711) = 0.1071$$

Update Input → Hidden Weights

For ***h1***

$$w_1^{new} = 0.1 - (0.1 \times -0.0088 \times 1) = 0.10088$$

$$w_2^{new} = 0.2 \quad (\text{since } x_2 = 0)$$

$$b_1^{new} = 0.1 - (0.1 \times -0.0088) = 0.10088$$

For ***h2***

$$w_3^{new} = 0.3 - (0.1 \times -0.0102 \times 1) = 0.30102$$

$$w_4^{new} = 0.4$$

$$b_2^{new} = 0.1 - (0.1 \times -0.0102) = 0.10102$$

Updated Weights (After 1st Iteration)

v1 =0.5039, v2 =0.6043, b3 =0.1071

*w*1=0.10088, w2=0.2 , *w*3=0.3010 w4=0.4

*b*1=0.10088, *b*2=0.1010

Repeat same for second input:

x1 = 0, x2 = 1 and Target(t) = 1

Repeat same for third input:

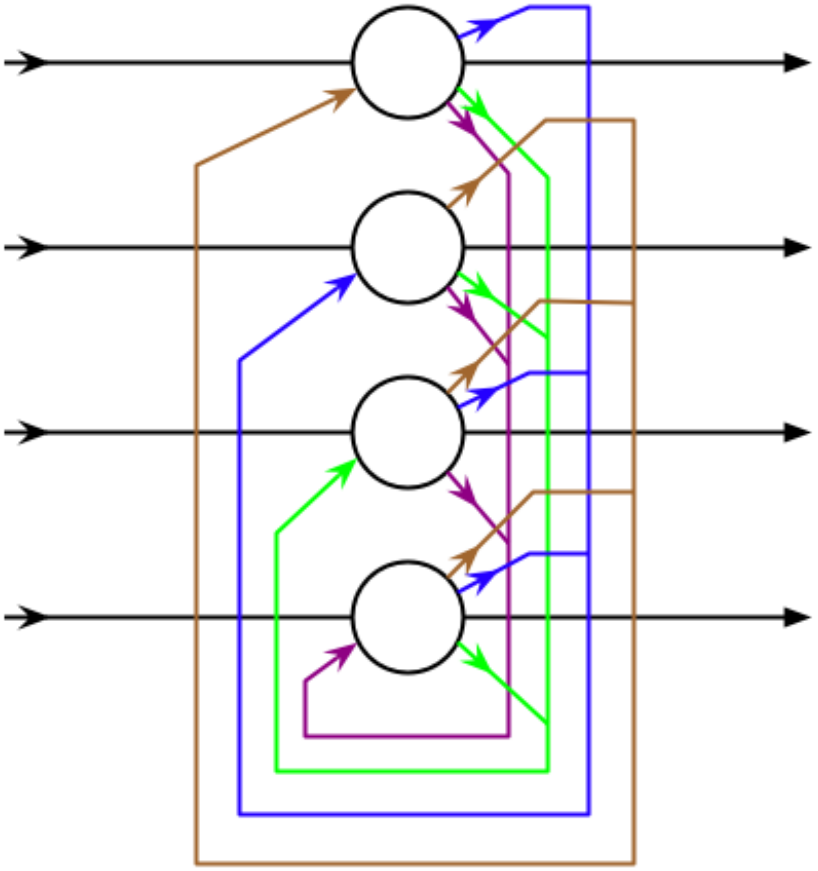
x1 = 1, x2 = 1 and Target(t) = 0

Repeat same for forth input:

x1 = 0, x2 = 0 and Target(t) = 0

Hopfield Network

- A Hopfield network is a form of recurrent artificial neural network popularized by John Hopfield in 1982, but described earlier by Little in 1974.
- The units in Hopfield nets are binary threshold units, i.e. the units only take on two different values for their states and the value is determined by whether or not the units' input exceeds their threshold. Hopfield nets normally have units that take on values of 1 or -1



Updating one unit (node in the graph simulating the artificial neuron) in the Hopfield network is performed using the following rule:

$$s_i \leftarrow \begin{cases} +1 & \text{if } \sum_j w_{ij} s_j \geq \theta_i, \\ -1 & \text{otherwise.} \end{cases}$$

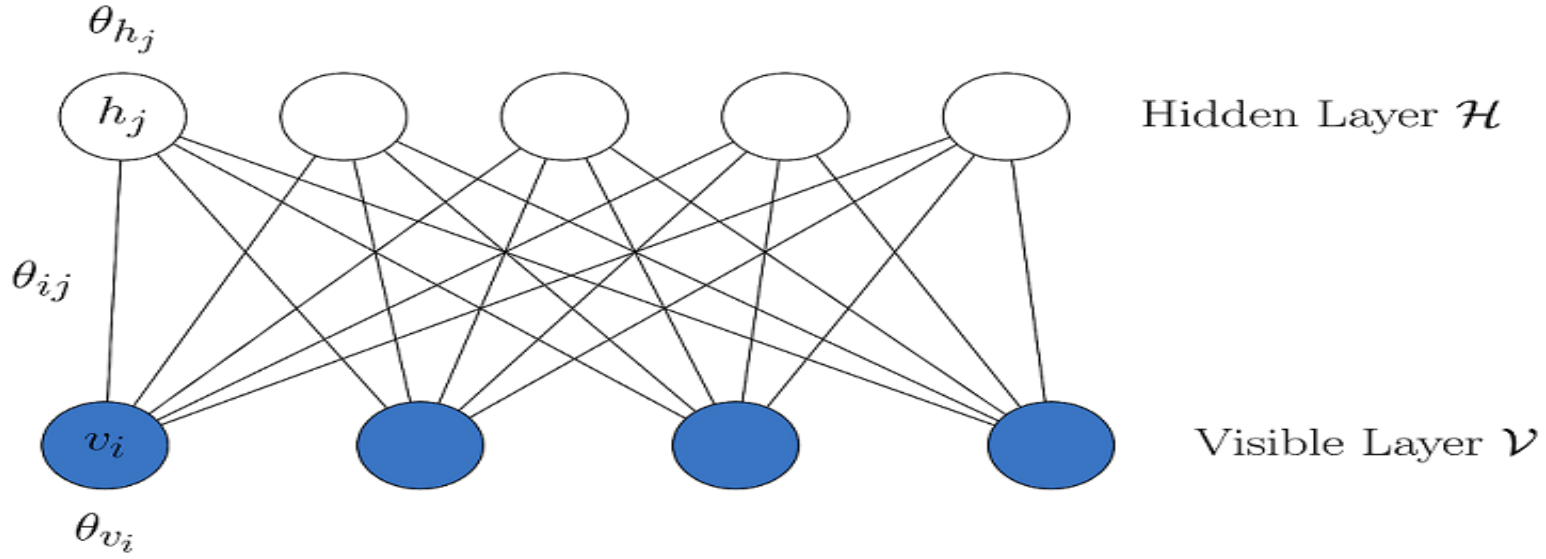
Where
S_i= is the state of unit j
W_{ij}= is the strength of the connection weight from unit j to unit i

θ_i =is the threshold of unit i.

Boltzmann Machines

- A Hopfield Net consisting of Binary Stochastic Neuron with hidden units is called Boltzmann Machine.
- A Boltzmann Machine is a network of symmetrically connected, neuron like units that make stochastic decisions about whether to be on or off.

Structure of Boltzmann Machine



- The stochastic neurons of Boltzmann machine are in two groups: visible and hidden.
- Visible neurons provide an interface between the net and its environment.

Structure of Boltzmann Machine

- During the training phase, the visible neurons are clamped; the hidden neurons always operate freely, they are used to explain underlying constraints in the environmental input vectors.
- The hidden units explain underlying constraints by capturing higher-order correlations between the clamping vectors.
- Boltzmann machine learning may be viewed as an unsupervised learning procedure for modeling a distribution that is specified by the clamping patterns.
- In Boltzmann Machine, everything is defined in terms of the energies of joint configuration of the visible and hidden units.

Applications of Boltzmann Machine

- Stock market trend prediction
- Character recognition
- Face recognition
- Internet application
- Cancer detection
- Loan application
- Decision making

Assignment

Explain different Types Of Activation Functions In Neural Networks.

Machine Vision and Future of AI

Explore Yourself

Thank You !!!